



Système IoT de Monitoring

Encadré par : Pr. BOUHADDOU Safae

Détection d'anomalies embarquée avec TinyML sur Raspberry Pi 4



FETTAH Saad



**CHEYOUKH
Boubker**



**ELATMANI
Abderrazzak**



**SALEK
Mehdi**



**KHOMANIA
Hamza**



**FARIS
Hamada**



The image shows a patient lying in a hospital bed at night. In the foreground, a medical device with a digital display shows '125 ml/h'. Above the patient, a large, futuristic, semi-transparent digital overlay displays 'REALTIME VITALS: PATIENT ID: A-214'. This overlay contains various charts and data points: Flow (125 ml/h), HR (74 bpm), SpO2 (98%), BP (128/82 mmHg), and Resp (16/min). To the right of these charts is a section titled 'The NEXUS HEALTH MONITOR' which includes bar charts and a list of values: Total Infused (485ml), Total Infused (485ml), Pda Rpm (515ml), Pda Rpm (515ml), and Temp Rpm (04:08h). The patient's arm has an IV line with a glowing green light at the insertion point. The background shows a typical hospital room with another monitor and medical equipment.

Contexte, Problématique

Contexte

Dans un environnement hospitalier, les **pompes** à perfusion jouent un rôle critique en assurant l'administration précise de médicaments et de fluides aux patients. Ces équipements critiques nécessitent une **surveillance** constante pour garantir la **sécurité** des patients.

Problématique



Comment concevoir un système IoT embarqué capable de détecter efficacement les anomalies liées aux pompes à perfusion, tout en minimisant la consommation énergétique et la charge réseau sur un Edge device — Raspberry Pi ?

Objectifs du projet

Cinq objectifs structurants, du capteur à la visualisation

01

**Collecter &
simuler
données IoMT
sur Raspberry
Pi 4**

02

**Comparer 3
strategies de
transmission
réseau
(S1,S2,S3)**

03

**Implémenter 3
modèles TinyML
de détection
d'anomalies**

04

**Mesurer la
consommation
énergétique à
chaque étape**

05

**Produire un
dashboard
Streamlit et un
rapport
complet**

Architecture IoT à 4 couches

Une décomposition standardisée du capteur à l'utilisateur final

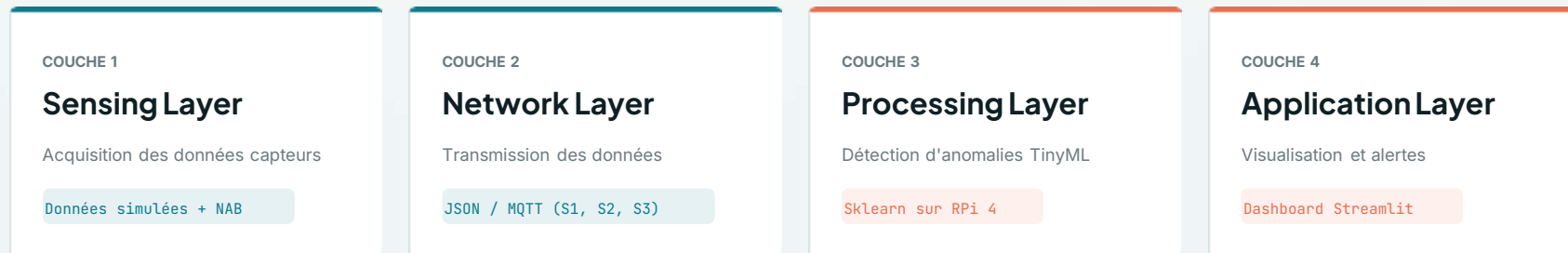


Tableau 1 – Architecture du système IoT à 4 couches

Sélection des datasets

Deux sources combinées pour 13 096 échantillons

SOURCE 1

Données simulées

1 000 échantillons

94 anomalies · taux 9.4%

Générées avec NumPy / Pandas pour modéliser les paramètres d'une pompe à perfusion

SOURCE 2

Dataset NAB

12 096 échantillons

108 anomalies · taux 0.9%

Numenta Anomaly Benchmark — exigence du module, 3 fichiers utilisés

TOTAL

Combiné

13 096 échantillons

202 anomalies · taux 1.5%

Volume final utilisé pour entraîner et évaluer les modèles TinyML

Données simulées

Paramètres NumPy / Pandas — pompe à perfusion

PARAMÈTRE	NORMALE	ANOMALIE	UNITÉ
Débit	80 ± 5	< 5 ou > 200	ml/h
Pression	100 ± 10	> 150	mmHg
Alerte	0 normal	1 anomalie	binaire

TOTAL

1000

NORMAUX

906

90.6%

ANOMALIES

94

9.4%

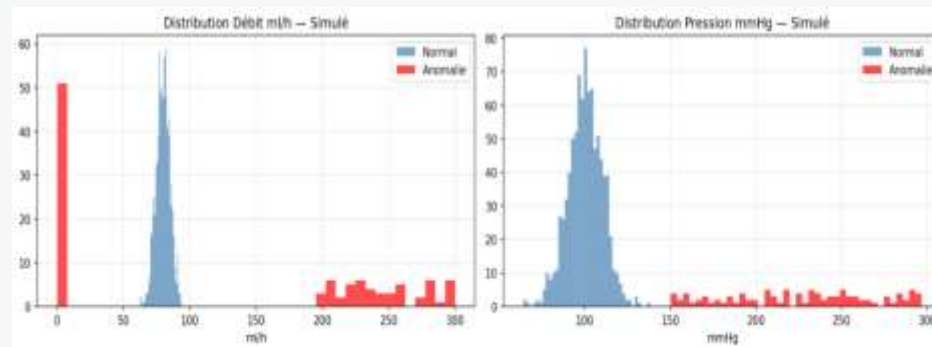


Figure 1 — Exploration des données simulées

Visualisation des anomalies

Distribution débit / pression et points d'alerte sur le jeu simulé

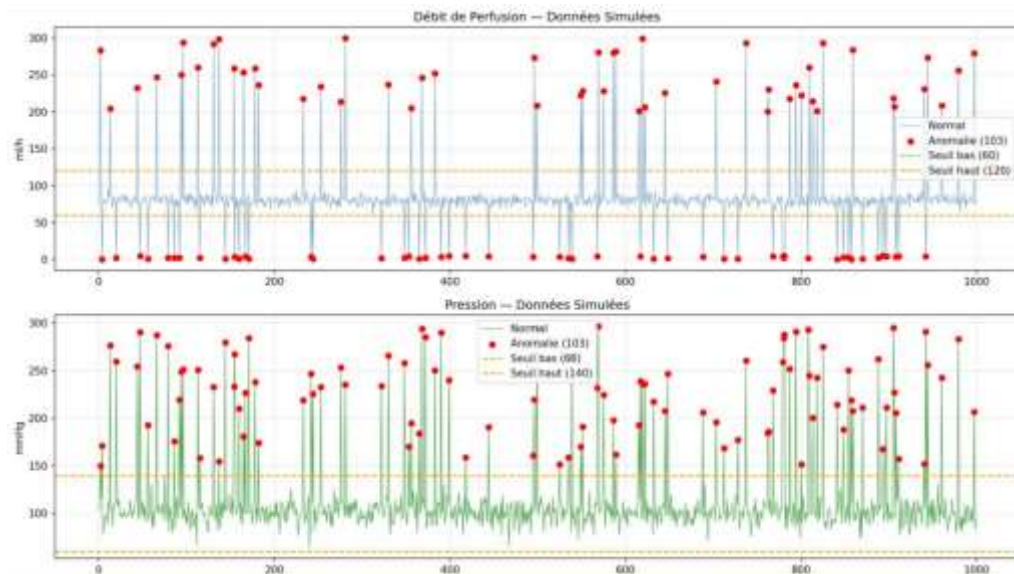


Figure 2 – Visualisation des anomalies · données simulées

Dataset NAB

Numenta Anomaly Benchmark — exigence officielle du module

POURQUOI NAB

Conformément aux exigences du module, le dataset NAB a été intégré comme source principale. Trois fichiers ont été utilisés pour enrichir la détection d'anomalies dans des séries temporelles réelles.

ÉCHANTILLONS

12 096

ANOMALIES

108

TAUX

0.9%

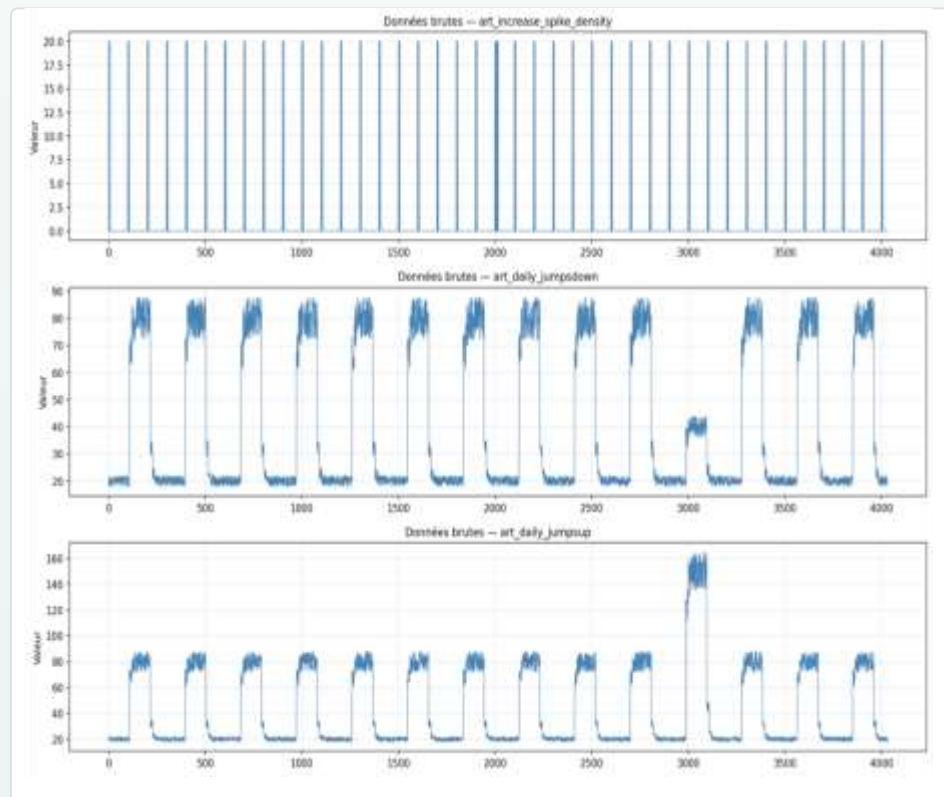
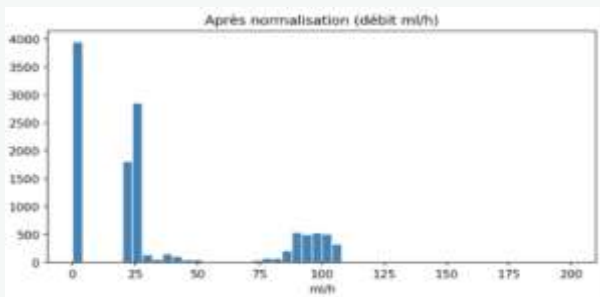
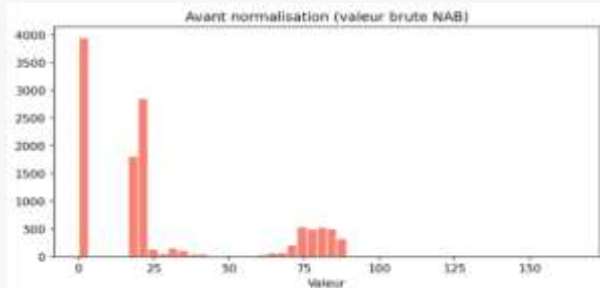


Figure 3 — Exploration des données brutes NAB

Pipeline de préparation des données

Normalisation · étiquetage · sélection des features · split train/test



ÉTAPE 1

Normalisation

Transformation et mise à l'échelle des séries NAB

ÉTAPE 2

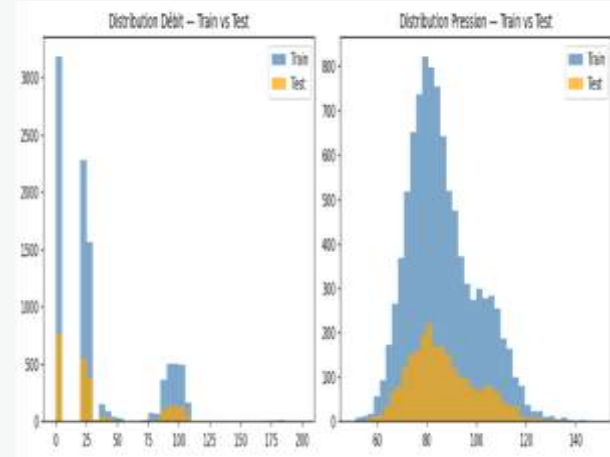
Étiquetage

On marque chaque donnée normale ou anomalie

```
=====
ÉTAPE 2 — ÉTIQUETAGE DES ANOMALIES
=====

📊 Statistiques débit :
Moyenne      : 34.93 ml/h
Écart-type   : 38.53
Seuil bas    : -42.14 ml/h
Seuil haut   : 112.00 ml/h

✅ Distribution des alertes :
alerte
0      11988
1       108
Name: count, dtype: int64
Taux anomalies : 0.89%
```



ÉTAPE 3

Features & split

On divise en 80% pour entraîner et 20% pour tester.

Entraînement

Évaluation des trois modèles TinyML

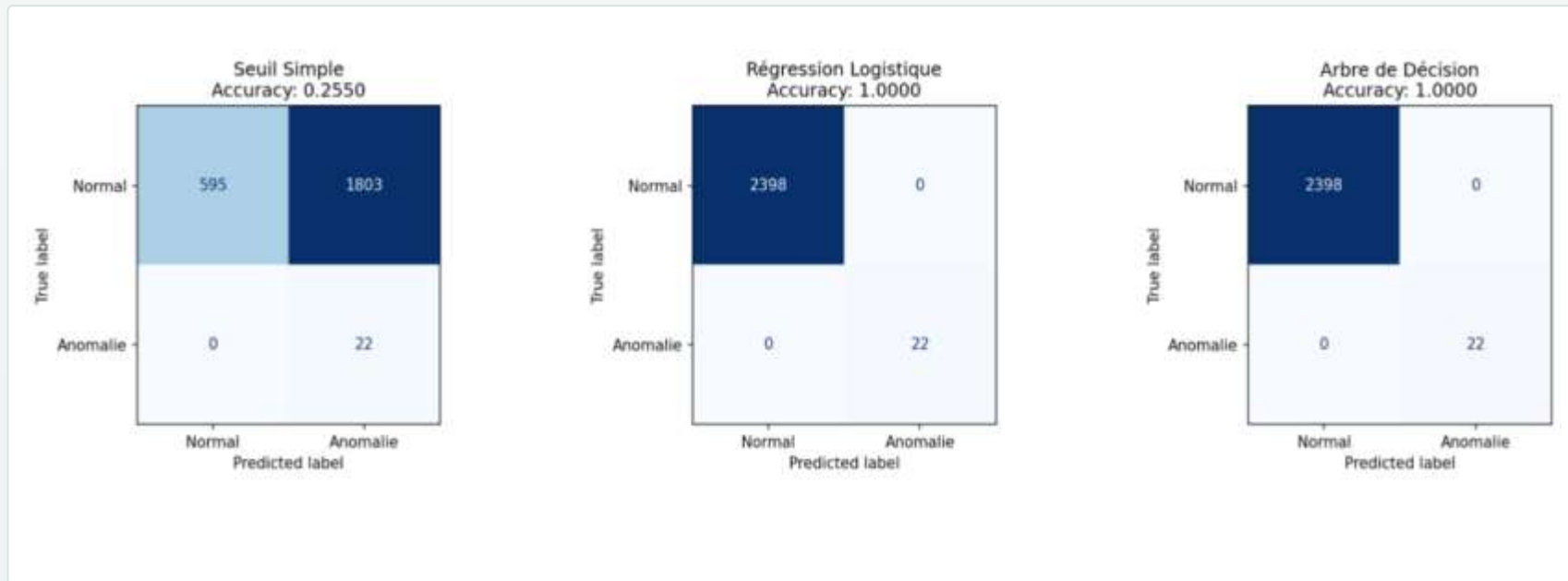


Figure 7 – Entraînement et évaluation des modèles

Benchmark

13

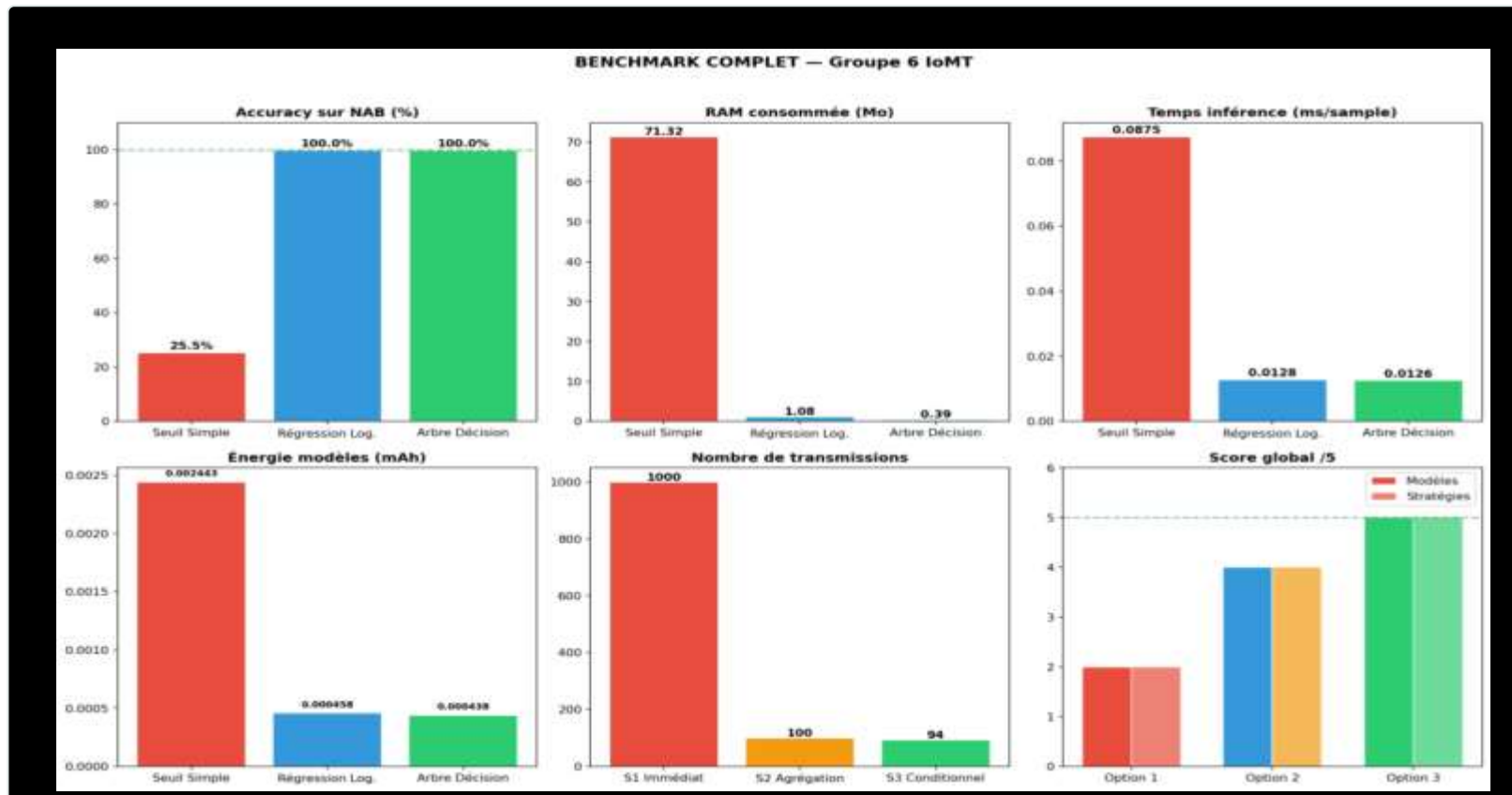


Figure 8 – Benchmark complet

Stack technique

Backend embarqué sur Raspberry Pi · Frontend React sur PC

● BACKEND · RASPBERRY PI

- **FastAPI**
API REST légère, rapide et performante
- **Modèles Python**
Traitement et analyse des données capteurs
- **JSON**
Mise à jour des données toutes les 2 secondes

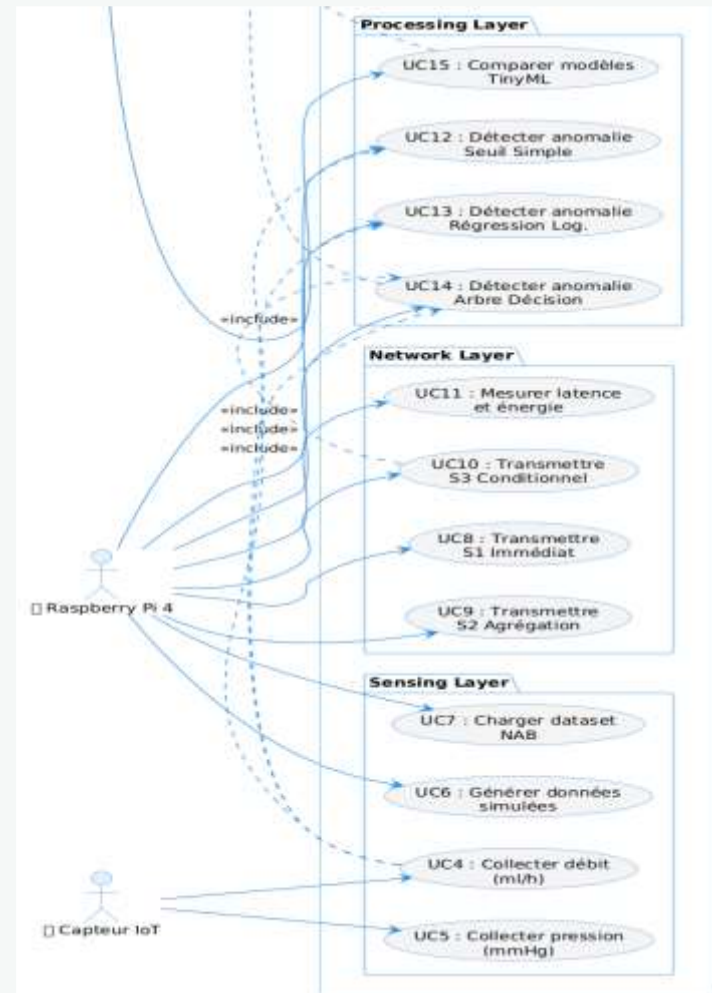
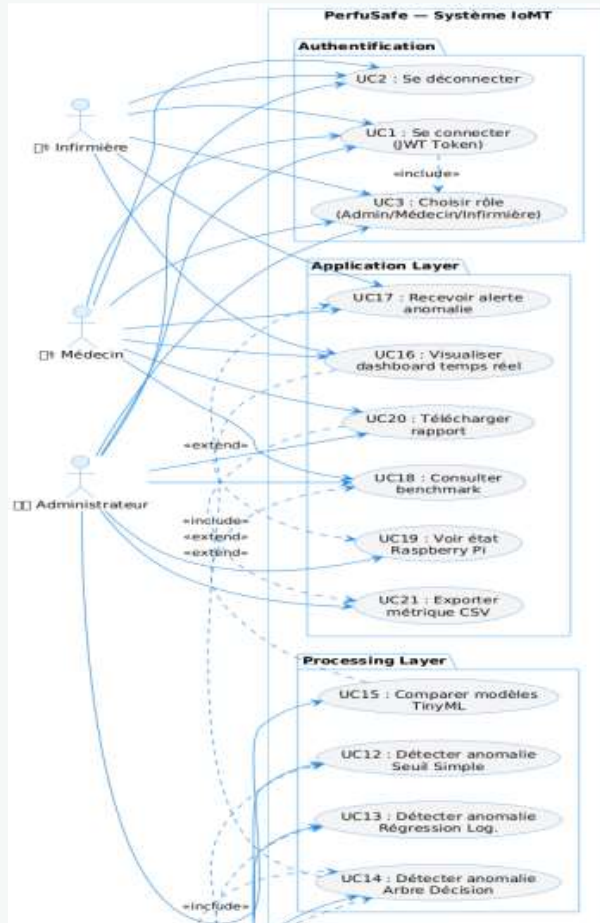
● FRONTEND · PC

- **Next.JS + TailwindCSS**
Interface moderne, responsive et professionnelle
- **Recharts**
Graphiques interactifs et animés
- **Firebase Auth**
Authentification Google Sign-In

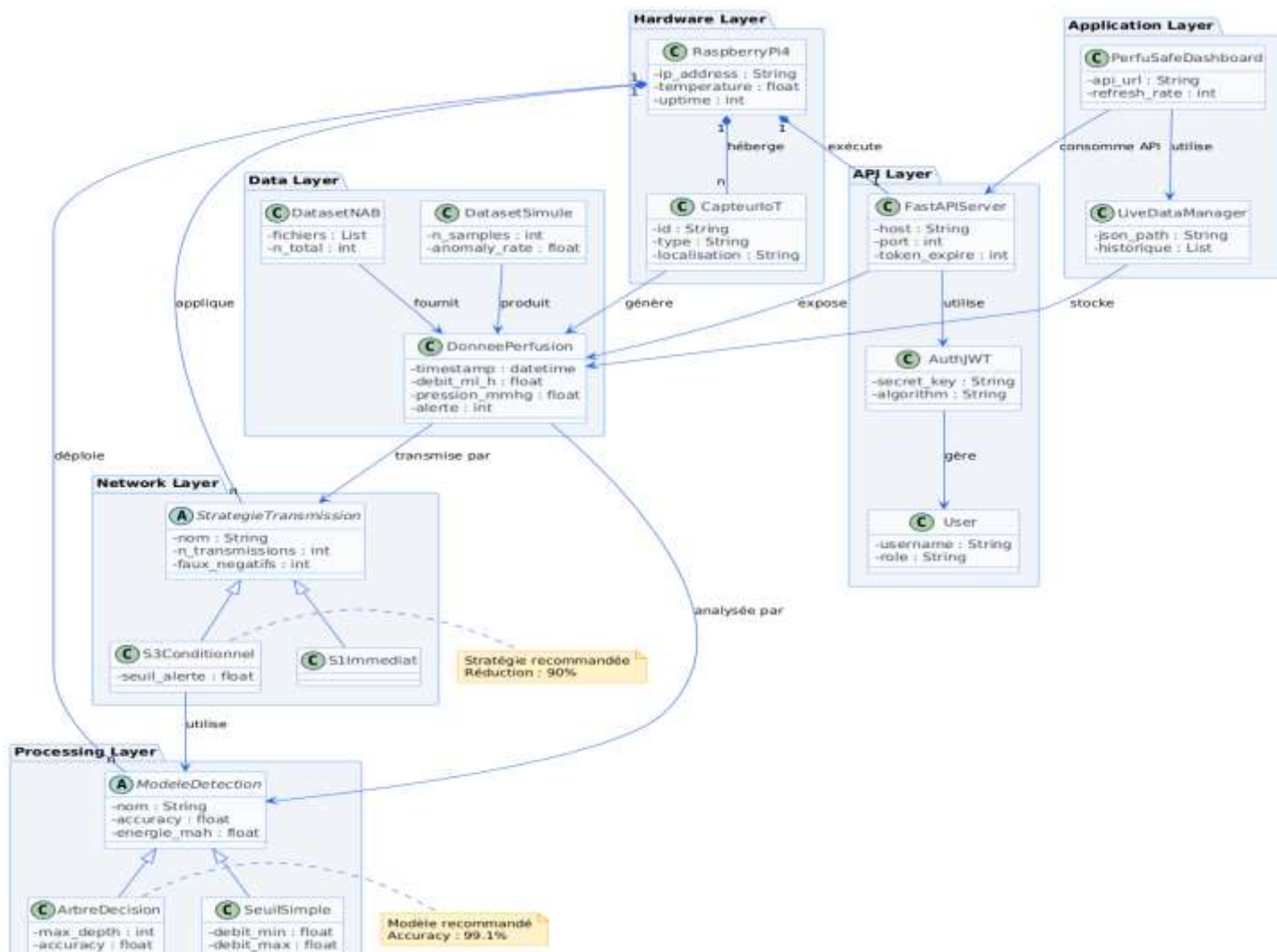
MODÉLISATION

Diagrammes UML

Use case



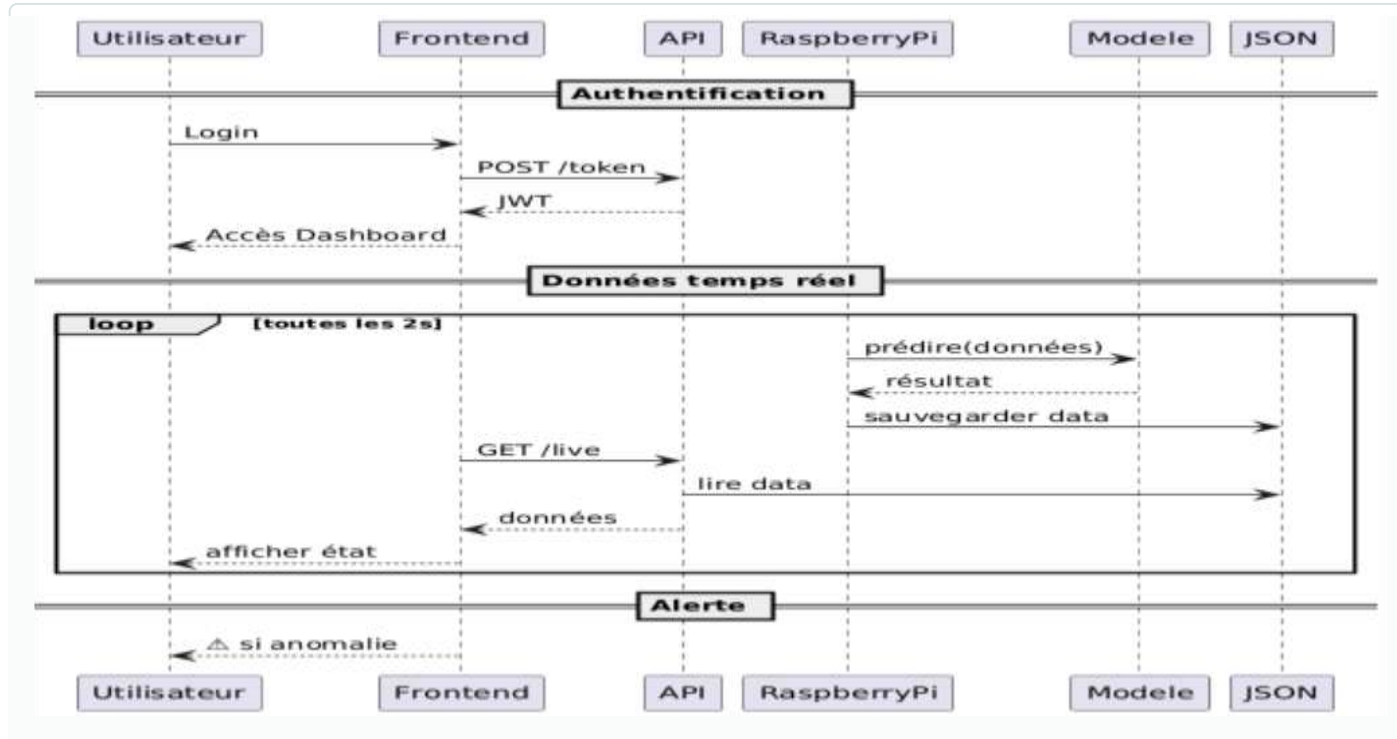
Diagrammes De classe



Diagrammes UML

18

Séquence



Configuration matérielle

Raspberry Pi 4 — environnement Python isolé

01 Raspberry Pi 4

Boîtier aluminium · 2 ventilateurs

02 Alimentation USB-C 27 W

Tension de référence pour la mesure énergétique

03 Raspberry Pi OS 64-bit

Contrainte obligatoire du module

04 Environnement virtuel

python3 -m venv iot_env

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installer la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations | https://aka.ms/PSWindows

PS C:\Users\bouda> ssh groupe6@groupe6-iot.local
The authenticity of host 'groupe6-iot.local (192.168.55.146)' can't be established.
ED25519 key fingerprint is SHA256:5+0UakdZ0H0uG4u79LSkZiyu+qum8UWUHL4dY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'groupe6-iot.local' (ED25519) to the list of known hosts.
groupe6@groupe6-iot.local's password:
Linux groupe6-iot 6.12.47+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1 (2025-09-16) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
groupe6@groupe6-iot:~$
```

Figure 12 — Connexion SSH au Raspberry Pi 4

MODÈLE 01

Seuil simple — Thresholding

Détection basée sur des seuils fixes. Modèle déterministe, aucun entraînement requis — la baseline absolue du benchmark TinyML.

DÉBIT

60 – 120

ml / h

PRESSION

60 – 140

mmHg

ACCURACY

99.1%

RAM

71.32 Mo

ÉNERGIE

0.0024

mAh

```
#!/usr/bin/env python3
import pandas as pd
import numpy as np
import time
import psutil
import os

# Chargement données
df = pd.read_csv("data/data_perfusion.csv")

# Seuils normalisés
DEBIT_MIN, DEBIT_MAX = 60, 120
PRESSION_MIN, PRESSION_MAX = 60, 140

# Mesure RAM avant
process = psutil.Process(os.getpid())
ram_avant = process.memory_info().rss / 1024 / 1024

# Inférence + mesure temps
start = time.time()
predictions = []
for _, row in df.iterrows():
    if (DEBIT_MIN <= row["debit_ml_h"] <= DEBIT_MAX and
        PRESSION_MIN <= row["pression_mmhg"] <= PRESSION_MAX):
        predictions.append(1)
    else:
        predictions.append(0)
end = time.time()

# Métriques
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
y_true = df["alerte"].values
y_pred = predictions

ram_apres = process.memory_info().rss / 1024 / 1024
temps_total = (end - start) * 1000
temps_par_inferece = temps_total / len(df)

print(f"n = {df.shape[0]}")
print(f"MODÈLE : - SEUIL SIMPLE")
print(f"n = {df.shape[0]}")
print(f"Accuracy : {accuracy_score(y_true, y_pred):.1f}%")
print(f"Temps inferece : {temps_par_inferece:.2f} ms/sample")
```

Figure 14 — Code du Modèle 1

MODÈLE 02

Régression logistique

Modèle linéaire supervisé entraîné sur 800 échantillons, soit 80% du jeu pour l'apprentissage et 20% pour le test. Très léger en mémoire vive.

TRAIN / TEST

80% / 20%

ÉCHANTILLONS

800 train

ACCURACY

99.1%

RAM

1.08 Mo

ÉNERGIE

0.0005

mAh

```
#!/usr/bin/env python3
# model2_logistic.py

import pandas as pd
import numpy as np
import time
import psutil
import os

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Chargement données
df = pd.read_csv("data/data_perfusion.csv")
X = df[['monoc_mls', 'pression_meng']]
y = df['alarme']

# Split train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Mesure RAM avant
process = psutil.Process(os.getpid())
ram_avant = process.memory_info().rss / 1024 / 1024

# Entraînement
model = LogisticRegression()
model.fit(X_train, y_train)

# Inférence + mesure temps
start = time.time()
y_pred = model.predict(X_test)
end = time.time()

# Métriques
ram_apres = process.memory_info().rss / 1024 / 1024
temps_total = (end - start) * 1000
temps_par_infereuce = temps_total / len(X_test)

print("-" * 50)
print("MODÈLE 2 - RÉGRESSION LOGISTIQUE")
print("-" * 50)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Temps infereuce: {temps_par_infereuce:.4f} ms/sample")
```

Figure 15 – Code du Modèle 2

MODÈLE 03 · CHAMPION

Arbre de décision

Arbre contraint à 3 niveaux (max_depth = 3) pour limiter la consommation mémoire. Meilleur compromis précision / ressources, retenu pour le déploiement final.

PROFONDEUR

max_depth = 3

INFÉRENCE

0.0126 ms

ACCURACY

99.1%

RAM

0.39 Mo

ÉNERGIE

0.0004

mAh

```
#!/usr/bin/env python3
import pandas as pd
import numpy as np
import time
import os
import pickle
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Chargement données
df = pd.read_csv('data/perfusion.csv')
X = df[['diastolic_bp', 'systolic_bp']]
y = df['status']

# Split train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Prévenir RAM avant
process = psutil.Process(os.getpid())
ram_avant = process.memory_info().rss / 1024 / 1024

# Entraînement (arbre à 3 niveaux)
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

# Inférence = mesure temps
start = time.time()
y_pred = model.predict(X_test)
end = time.time()

# Métriques
ram_apres = process.memory_info().rss / 1024 / 1024
temps_total = (end - start) * 1000
temps_par_inferece = temps_total / len(X_test)

print(f"RAM avant : {ram_avant} Mo")
print(f"RAM après : {ram_apres} Mo")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Temps inferece : {temps_par_inferece:.4f} ms/sample")
```

Figure 16 — Code du Modèle 3

Résultats — simulées vs NAB

Comparaison des trois modèles TinyML

DONNÉES SIMULÉES

MODÈLE	ACC.	PRÉC.	RECALL	RAM
Seuil	100%	1.00	1.00	71.32 Mo
Régression	100%	1.00	1.00	1.08 Mo
Arbre	100%	1.00	1.00	0.39 Mo

Tableau 4 — Résultats sur données simulées

DATASET NAB

MODÈLE	ACC.	TEMPS	ÉNERGIE	RAM
Seuil	99.1%	0.087 ms	0.0024	71.32
Régression	99.1%	0.013 ms	0.0005	1.08
Arbre	99.1%	0.013 ms	0.0004	0.39

Tableau 5 — Synthèse sur dataset NAB

Recommandation : l'arbre de décision pour le déploiement final — RAM minimale, inférence la plus rapide, énergie la plus faible.

Consommation énergétique

Mesure psutil — tension de référence 5V USB-C

MODÈLE 1 — SEUIL

9.36 ms · 0.044 J

0.0024 mAh

MODÈLE 2 — RÉGRESSION

2.75 ms · 0.008 J

0.0005 mAh

MODÈLE 3 — ARBRE

2.63 ms · 0.008 J

0.0004 mAh

STRATÉGIE S3 — CONDITIONNEL

9.75 ms · 0.029 J

0.0016 mAh

```
=====
TABLEAU RÉCAPITULATIF - SENSING LAYER
=====
```

Type collecte	mAh	Joules
Génération simulée	0.003991	0.071830
Lecture NAB	0.022890	0.412014
Collecte + Prétraitement	0.005664	0.101949

```
=====
BILAN ÉNERGÉTIQUE GLOBAL DU SYSTÈME
=====
```

Sensing (collecte+prétraitement)	: 0.005664 mAh
Processing (Arbre de Décision)	: 0.000438 mAh
Network (S3 Conditionnel)	: 0.001625 mAh

TOTAL par cycle (100 samples)	: 0.007727 mAh
Estimation sur 1 heure	: 0.2318 mAh/h
Estimation sur 24 heures	: 5.56 mAh/jour

```
(iot_env) groupe6@groupe6-iot:~/projet_iot $
```

Figure 22 — Bilan énergétique global

Optimisations énergétiques

Trois optimisations cumulables — gain total 18.8%

OPTIMISATION 01

+5.6%

Mode Sleep

CPU idle entre les mesures pour réduire la consommation moyenne

OPTIMISATION 02

+3.4%

Quantification int8

Conversion float64 vers int8 pour des opérations moins gourmandes

OPTIMISATION 03

+11.0%

Fréquence adaptative

Réduction des collectes inutiles selon le contexte

GAIN TOTAL COMBINÉ

de 0.0078 à 0.0063 mAh / cycle

18.8%

```
(lat_ens) grupe@grupe-lot:~/grupe_lot$ python optimisation_energie.py
=====
OPTIMISATIONS ÉNERGETIQUES - GROUPE 6
=====

➤ Optimisation 1 - Mode Sleep entre collectes
Principe : CPU passe en idle entre deux mesures
Sans sleep : courant 645 mA - 0.001620 mAh
Avec sleep : courant 609 mA - 0.001529 mAh
Gain estimé : 5.6%

➤ Optimisation 2 - Quantification modèle (Float64 → int8)
Float64 : 2.127 ms - 0.000301 mAh - Accuracy 1.0000
int8 : 2.104 ms - 0.000368 mAh - Accuracy 1.0000
Gain : 3.4%

➤ Optimisation 3 - Fréquence de collecte adaptative
Principe : réduire nb de collectes/min quand tout est normal
Comparaison sur une fenêtre de 60 secondes fixe
Fixe : 300 col/min - CPU 15% - 12.2668 mAh
Adaptatif : 20 col/min - CPU 6% - 10.9998 mAh
Gain estimé : 11.0%

=====
BILAN DES OPTIMISATIONS
=====
Optimisation      Gain
-----
1. Mode sleep (CPU idle entre mesures)  5.6%
2. Quantification int8                  3.4%
3. Fréquence adaptative                 11.0%
-----
Gain total combiné  18.8%

Sans optimisation : 0.007727 mAh/cycle
Avec optimisation : 0.006379 mAh/cycle

➤ Recommandations Finales :
1. Mode sleep → gain 5.6% (CPU idle entre mesures)
2. Quant. int8 → gain 3.4% (inférence plus légère)
3. Freq. adapt → gain 11.0% (moins de collectes inutiles)

=====
Gain total estimé : 18%
(lat_ens) grupe@grupe-lot:~/grupe_lot$
```

Figure 26 – Résultats des optimisations

Stratégies de transmission réseau

Comparaison de S1, S2 et S3 sur le Raspberry Pi 4

S1 Envoi immédiat Chaque donnée transmise dès collecte. Latence minimale, consommation maximale. TRANSMISSIONS 1000 ÉNERGIE 0.0021 Réduction 0%	S2 Agrégation N=10 Buffer de 10 mesures avant envoi groupé. Forte réduction du trafic réseau. TRANSMISSIONS 100 ÉNERGIE 0.0018 Réduction 90%	S3 RECOMMANDÉE Envoi conditionnel Seules les anomalies détectées localement sont transmises. Optimal pour Edge hospitalier. TRANSMISSIONS 94 ÉNERGIE 0.0016 Réduction 90.6%
--	---	---

Tableau 7 – Comparaison des stratégies de transmission

Dashboard Streamlit

Cinq onglets fonctionnels — accessible via navigateur

TAB 1

KPIs Temps Réel

Métriques globales · graphiques
débit / pression

TAB 2

Comparaison Modèles

Tableaux et graphiques simulé vs
NAB

TAB 3

Stratégies Réseau

Comparaison S1 / S2 / S3 et énergie

TAB 4

Alertes Temps Réel

Détection live · refresh auto 2 s

TAB 5

Rapport

Export .txt et .csv téléchargeables



Figure 32 — KPIs temps réel



Figure 34 — Stratégies réseau

Alertes temps réel

Détection live avec prédictions des trois modèles en parallèle

ACQUISITION

Capteurs simulés

Génération en continu des paramètres pompe

INFÉRENCE

3 modèles en parallèle

Seuil · Régression · Arbre — comparaison instantanée

NOTIFICATION

Refresh auto 2 s

Toute anomalie détectée remonte instantanément

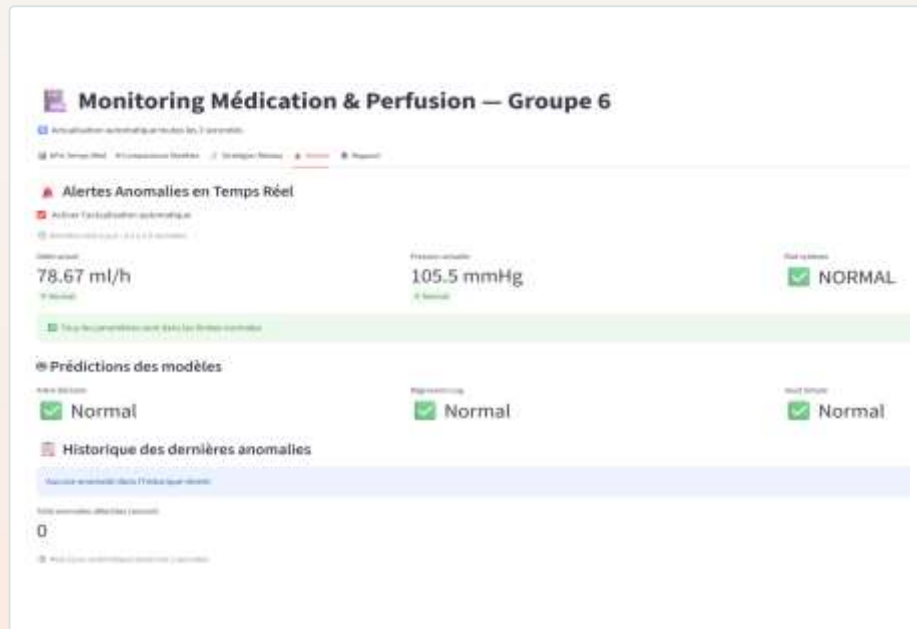
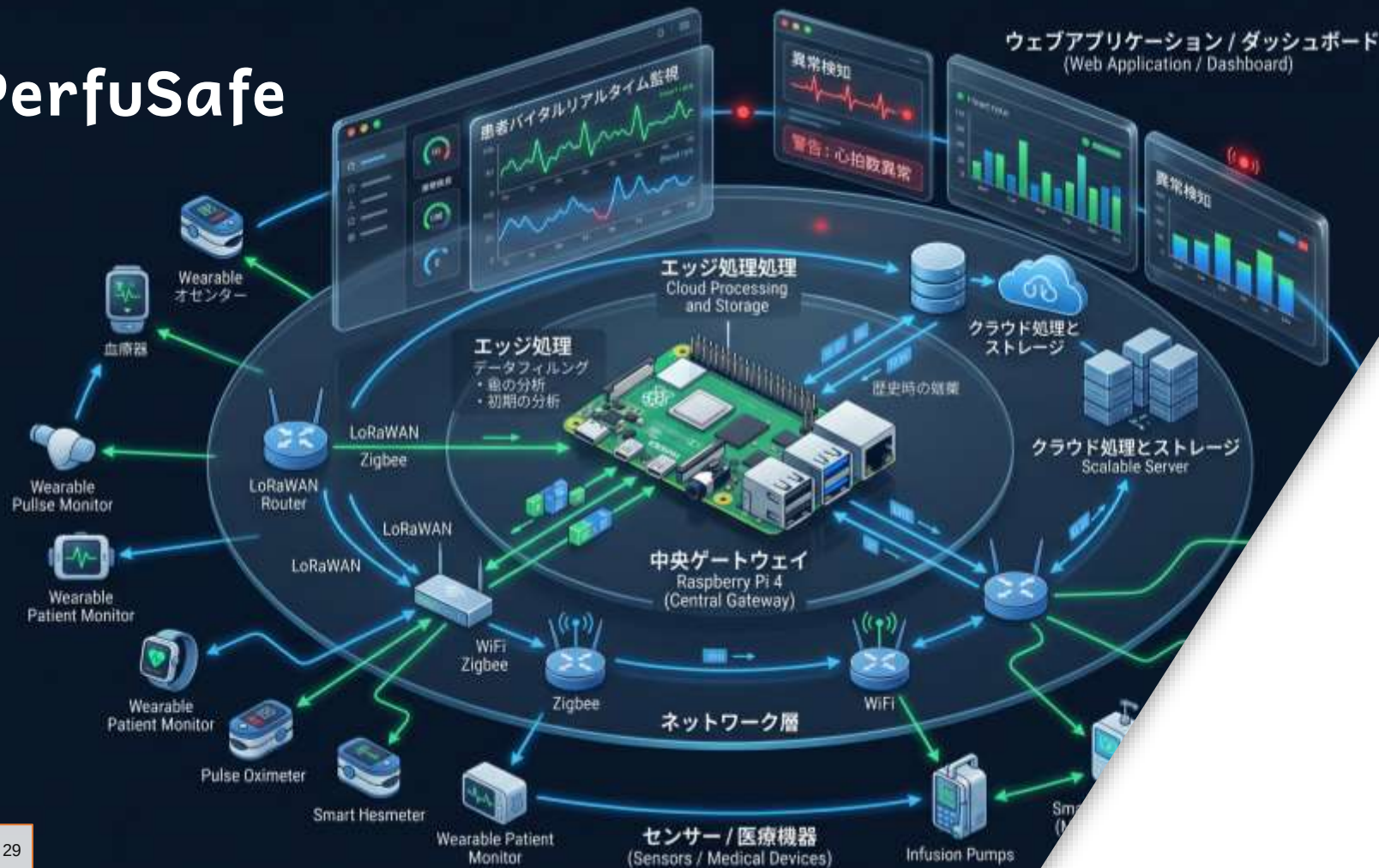


Figure 36 — Onglet Alertes · prédictions des 3 modèles

PerfuSafe



APPLICATION WEB

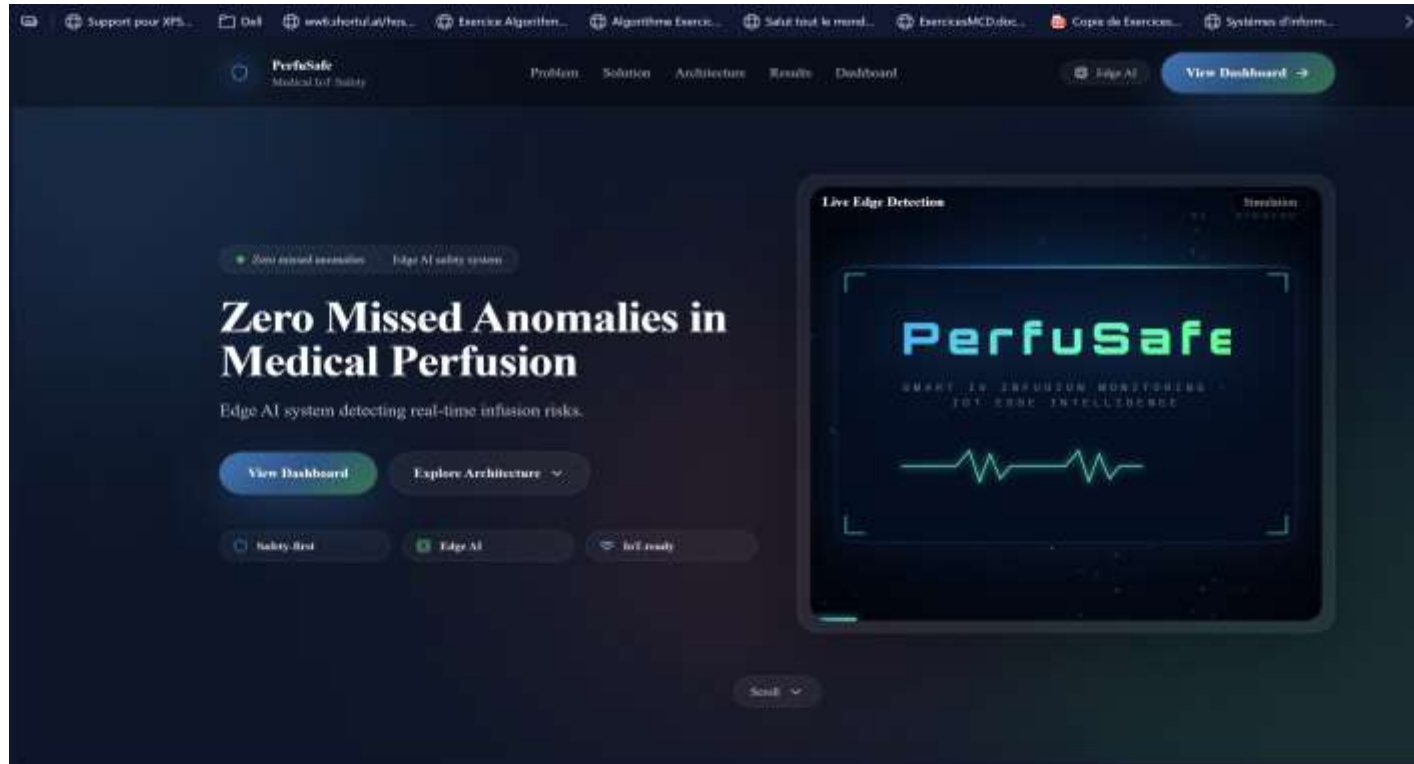
PerfuSafe

Backend : FastAPI sur Raspberry Pi



Figure 39 – Swagger UI

FastAPI · REST



APPLICATION WEB

Frontend(Next.js + Firebase)

Combinaison optimale & validation

Toutes les exigences du module ont été satisfaites

COMBINAISON RECOMMANDÉE

01 Arbre de Décision

0.000438 mAh

02 Stratégie S3 conditionnel

0.001625 mAh

03 Sleep + int8 + fréquence adaptative

Gain énergétique cumulé de 18.8%

Consommation totale par cycle

0.006274 mAh

VALIDATION DES EXIGENCES

Raspberry Pi OS 64-bit	Opérationnel
Environnement virtuel Python	iot_env actif
Dataset NAB intégré	12 096 lignes
3 modèles TinyML comparés	100% / 99.1%
3 stratégies de transmission	S3 recommandée
Métriques énergétiques	Toutes mesurées

PERSPECTIVES & AMÉLIORATIONS FUTURES

01

- LSTM et Autoencoder pour la détection d'anomalies
- l'apprentissage profond

02

- TensorFlow Lite Micro pour l'optimisation
- le déploiement sur systèmes embarqués

03

- ESP32 pour l'exécution en temps réel
- la connectivité IoT médicale



MERCI POUR VOTRE écoute

Boubker Cheyoukh | Saad Fettah | Abderrazzak Elatmani |
Mehdi Salek | Hamaza khomania | FARIS Hamada

Encadré par : Pr. BOUHADDOU Safae